

WES 237C: Project 1 – FIR Filter Design

Author: Genaro Salazar Ruiz

Date: 10/08/2026

GitHub: [Link](#)

Question 1 – FIR11 Baseline

(a) Performance Metrics

- **Latency:** 19 clock cycles
- **Initiation Interval (II):** 20 clock cycles
- **Loop Iteration Latency:** 16 clock cycles -> (Loop II = 1)
- **Throughput:**

$$Throughput = \frac{1}{20 \times 6.912ns} = 7.23MHz$$

(b) Resource Utilization

Resource	Used	Available	Utilization (%)
BRAM_18K	0	280	0%
DSP	2	220	~0%
FF	733	106,400	~0%
LUT	383	53,200	~0%

FIR 128 Base Code

```
static data_t shift_reg[N]; //static array for sample persistence
acc_t acc = 0; //accumulaor
//shift loop
```

```

for(int i = N-1; i >= 0; i--) //loop backwards
{
    if(i){ //if not index zero
        shift_reg[i] = shift_reg[i-1];
        acc += shift_reg[i] * c[i];
    }
    else //is 0
    {shift_reg[0] = x; //
        acc += x * c[0];
    }
}
*y = (data_t)acc;

```

Question 2 – Variable Bitwidths

(a) Bitwidth Combination Table

Config	coef_t Bitwidth	acc_t Bitwidth	Latency (cycles)	II(cycles)	Est. Clock (ns)	Throughput (MHz)	BRAM	DSP	FF	LUT
Base	32-bit	32-bit	135	136	6.912	1.063	3	2	610	305
Intermediate	ap_int<16>	ap_int<16>	134	135	6.712	1.103	2	1	244	239
Min.	ap_int<5>	ap_int<16>	134	135	6.712	1.103	2	1	244	239

(b) Minimum Bitwidth Analysis

- Minimum Bitwidth for coef_t: ap_int<5>
- Minimum Bitwidth for acc_t: ap_int<16>
- **Verification:** The testbench still passed when the bit width was set to 5 for coef_t and 16 for acc_t.

Question 3 – Pipelining

(a) Baseline FIR128 Metrics

- **Latency:** 135 cycles
- **II:** 136 cycles

- **Estimated Clock:** 6.912 ns
- **Resource Usage:** BRAM: 3, DSP: 2, FF: 610, LUT: 305

(b) Pipeline Disabled

- **Latency:** 641 cycles
- **II:** 642 cycles
- **Estimated Clock:** 6.912 ns
- **Resource Usage:** BRAM: 2, DSP: 2, FF: 412, LUT: 279

(c) Manual Loop Initiation Interval Sweep Table

Loop Pragma	Overall Latency (cycles)	Overall II (cycles)	Est. Clock (ns)	Throughput (MHz)	BRAM	DSP	FF	LUT
II = 1	135	136	6.912	1.064	3	2	610	305
II = 2	262	263	6.912	0.55	2	2	513	321
II = 3	389	390	6.912	0.371	2	2	480	318
II = 4	516	517	6.912	0.279	2	2	480	323
II = 5	643	644	6.912	0.225	2	2	453	300
pipeline off	641	642	6.912	0.225	2	2	412	279

(d) Saturation Limit for II

- **Value:** 5 cycles.
- **Explanation:** Increasing the target initiation interval beyond the unpipelined iteration latency of the loop body yields no further significant resource-sharing benefit, as the loop approaches the performance of sequential execution.

(e) Default Loop Pipelining Behavior

- **Observation:** When no pipeline pragma is specified, Vitis automatically sets the pipeline target to $II = 1$ for inner loops. This is visible by seeing the identical performance between the baseline and manual setting of 1.

Question 4 – Removing Conditional Statements

(a) Pipeline Comparison

Architecture	Latency (cycles)	II (cycles)	Est. Clock (ns)	Throughput (MHz)	BRAM	DSP	FF	LUT
With Conditionals	135	136	6.912	1.064	3	2	610	305
Without Conditionals (Hoisted)	134	135	6.912	1.072	2	2	416	316

(b) No-Pipeline Comparison

Architecture	Latency (cycles)	II (cycles)	Est. Clock (ns)	Throughput (MHz)	BRAM	DSP	FF	LUT
With Conditionals	641	642	6.912	0.225	2	2	412	279
Without Conditionals (Hoisted)	636	637	6.912	0.227	2	2	345	270

- **Analysis:** Removing the conditional statement if(i) hoists the boundary condition out of the loop body. Based on the output, this eliminates at least 1 cycle of multiplexing and loop delay. However, if the loop was parallelized, the cycle-saving of this approach would be cascaded, resulting in greater optimization.

Question 5 – Loop Partitioning

(a) Partitioning Description

- **Opportunity:** In the original loop, shifting array values and computing the accumulator create contention for the same memory ports. By separating them into two dedicated loops, one for shifting and one for accumulating, the loops can run separately using separate memory, improving throughput.

(b) Comparison With vs. Without Loop Partitioning

Design	Latency (cycles)	II (cycles)	Est. Clock (ns)	Throughput (MHz)	BRAM	DSP	FF	LUT
Without Partitioning (Q4 Hoisted)	636	637	6.912	0.227	2	2	345	270
With Loop Partitioning	265	266	6.912	0.544	3	2	398	414

(c) Loop Partitioning with Unrolling

Design	Latency (cycles)	II (cycles)	Est. Clock (ns)	Throughput (MHz)	BRAM	DSP	FF	LUT
Partitioned + Unrolled	218	219	6.978	0.654	2	54	12849	10271

(d) Relationship Between Unrolling and Pipelining

- **Loop unrolling:** Does multiple calculations at once by duplicating hardware, making the design faster but using more resources.
- **Pipelining:** Overlaps different loop iterations so the hardware can keep working instead of waiting for each iteration to finish.
- **Using both:** Unrolling processes more data per clock cycle, while pipelining keeps new data flowing through the hardware. However, without splitting memory into multiple banks, BRAM port limits can prevent the design from accessing enough data at once.

Question 6 – Memory Partitioning

(a) Array Partitioning Exploration Table

Partitioning Strategy	Factor	Latency (cycles)	II (cycles)	Est. Clock (ns)	Throughput (MHz)	BRAM	DSP	FF	LUT
Default (BRAM) HLS Unroll	2	218	219	6.978	0.654	2	54	12849	10271
Block Partitioning	2 / 4	208	209	7.040	0.680	8	8	1899	2119
Cyclic Partitioning	2 / 4	108	109	6.923	1.325	8	8	1244	930
Complete Partitioning	128	107	108	7.006	1.321	0	8	5406	12035

- **Best Performance:** Complete partitioning provides the highest throughput because it breaks down both shift_reg and c arrays into individual registers, which allows all 128 taps to be accessed simultaneously in a single clock cycle.

(b) Interdependence of Unrolling and Memory Partitioning

- **Unrolling without Partitioning:** The loop is unrolled, but memory accesses remain serialized because standard BRAM blocks have only two read/write ports. Hence, the tool cannot schedule the operations in parallel, which causes high latency and fails pipeline constraints.
- **Partitioning without Unrolling:** Memory ports are expanded into discrete registers, but a sequential rolled loop only reads one element per cycle, which wastes the available memory bandwidth and consumes extra registers without any performance gain.

Question 7 – Best Architecture

(a) Optimal Design Configuration & Performance

- **Optimizations Applied:**
 - Complete loop and memory partitioning for shift_reg and c
 - Full loop unrolling for both shift and accumulator loops
 - Full loop unrolling sets II = 1
 - Bit width optimization in fir.h
- **Performance Summary:**
 - **Latency:** 27 clock cycles
 - **Initiation Interval (II):** 28 clock cycles

- **Estimated Clock Period:** 6.603 ns
- **Throughput:** 5.41 MHz

(b) Resource Usage & Trade-Off Analysis

Resource	Used	Available	Utilization (%)
BRAM_18K	0	280	0%
DSP	1	220	~0%
FF	3181	106,400	2
LUT	2200	53,200	4

•

Analysis:

- **Trading Space for Speed:** Instead of sharing a single memory block and calculating taps one by one step-by-step, my final design used 128 mini-calculators working all at once in parallel. That's why hardware resource usage goes way up, but it allows the chip to crunch the entire filter output really fast.
- **Pipeline Targets vs. Unrolling:** While you can explicitly set pipeline targets for standard loops, full unrolling inherently assumes a pipeline target of 1. Because unrolling completely flattens the loop into parallel hardware gates rather than sequential steps, it bakes an Initiation Interval of 1 straight into the hardware design. Specifying both `#pragma` directives in the loop will break the program at runtime.

Conclusion

This project highlights how HLS bridges software intent and hardware reality. Achieving maximum throughput is not a single fix, but a co-optimization chain that involves shrinking bitwidths to cut redundant multiplier size, code hoisting and loop partitioning to remove control hazards, and pairing full loop unrolling with complete memory partitioning to unlock parallel combinational execution. This goes to show how a well-thought-out architecture trades silicon area for maximum sample rate.